

ESIC HMS — System Design & Capacity Plan

ESIC HMS — System Design & Capacity Plan

Technical profile and infrastructure capacity plan for the ESIC Hospital Management System. Figures are engineering estimates derived from the stated assumptions in §3, using standard capacity-planning methodology (Little's Law for throughput, per-entity storage modeling for data growth). Live production traffic data is not yet available; these figures are intended to guide initial infrastructure sizing and should be recalibrated against real usage once the system is live.

Prepared: 2026-09-17.

Executive Summary

System profile

Backend	NestJS 10 + Prisma 5 on Node.js 20, Express adapter
Frontend	React 18 + Vite 5 + Tailwind, static SPA build
Database	PostgreSQL 15 — 59 data models, 11 schema migrations
Cache layer	Redis 7 (reserved for queue/session/cache use as the system scales)
Codebase size	≈35,200 lines of code across 239 files, 21 backend domain modules
PDF/document generation	Server-side rendering via a managed headless-browser instance
CI pipeline	Automated lint, type-check, and test execution on every change

Capacity baseline — single hospital (assumptions detailed in §3)

Daily clinical volume	600 OPD visits, 24 new IPD admissions (120-bed capacity, 80% occupancy)
System users	180 registered accounts, 150 daily active users, ~45 peak concurrent
Request volume	≈97,200 requests/day (≈10,800 writes, ≈86,400 reads)
Throughput	Average ~2.7 RPS, peak ~10–14 RPS, peak database load ~40–56 QPS
Storage per encounter	≈19 KB per OPD visit, ≈100 KB per IPD admission (inclusive of transaction audit records)
Storage growth	≈5 GB/year, ≈25 GB projected over 5 years
Bandwidth	≈290 MB/day, ≈8.7 GB/month
Memory footprint	API tier: 1.5–2 GB per instance; database tier: 2–4 GB
Cache working set	Under 50 MB for master/reference data

Multi-hospital / platform-admin expansion — scale outlook

Metric	1 hospital	10 hospitals	50 hospitals
Daily active users	150	1,500	7,500
Peak RPS	~12	~120	~600
Peak QPS (database)	~48	~480	~2,400
Storage growth/year	~5 GB	~50 GB	~250 GB
Storage at 5 years	~25 GB	~250 GB	~1.25 TB
Bandwidth/month	~9 GB	~90 GB	~450 GB

The platform architecture scales linearly with the number of participating hospitals at both throughput and storage layers; §11 details the data-model and access-control extensions required to operate a multi-hospital deployment under a single platform-level administrative tier.

1. Technology Stack

Layer	Technology	Version	Notes
Backend runtime	Node.js	20.x	LTS release
Backend framework	NestJS	10.4.15	Express adapter
ORM / data layer	Prisma	5.22.0	Code-first schema, migration-managed
Authentication	@nestjs/jwt , passport-jwt , bcryptjs	10.2.0 / 4.0.1 / 2.4.3	Access token (8-hour validity) + refresh token (7-day validity)
Task scheduling	@nestjs/schedule	4.1.2	In-process scheduled job for daily IPD finance processing
Document rendering	Puppeteer (managed headless Chromium)	25.9.0	Shared browser instance for receipt/report PDF generation
QR code generation	qrcode	1.5.4	In-memory generation, no external storage dependency
Input validation	class-validator , class-transformer	0.14.1 / 0.5.1	Global request validation pipeline
Frontend framework	React	18.3.1	With react-router-dom 6.28.1
Frontend build tool	Vite	5.4.11	Environment configuration resolved at build time
Styling	Tailwind CSS	3.4.17	
Data visualization	Recharts	3.10.0	Dashboard/analytics charts
Animation	Framer Motion	12.42.2	
Database	PostgreSQL	15	Relational, ACID-compliant
Cache / queue	Redis	7	Provisioned for session, queue, and caching workloads
Monorepo tooling	pnpm 9.15.4, Turborepo 2.3.3		
CI	GitHub Actions		Automated lint, type-check, and test on every push

2. Application Size

Item	Measurement
API source	171 TypeScript files, ~15,850 lines of code
Web source	68 TypeScript/TSX files, ~19,342 lines of code
Total first-party source	≈35,200 LOC across 239 files
Backend domain modules	21 (employee, patient, visit, OPD, admission, pharmacy, inventory, procurement, billing, catalog, laboratory, therapy, analytics, reports, dashboard, user, auth, facility, benefit, prescription, RBAC administration)
Database schema	59 Prisma models, 11 applied migrations
Dependency footprint	≈595 MB installed packages (includes bundled Chromium runtime for document rendering)
Compiled API build output	1.87 MB
Seeded RBAC configuration	13 system roles with a granular resource × action permission matrix

3. Capacity Planning — Baseline Assumptions

Inputs for a single mid-size hospital deployment. Adjust these to match a specific facility's actual profile.

Parameter	Value	Basis
OPD visits / day	600	Typical mid-size hospital OPD load
IPD beds	120 (80% occupancy → 96 occupied)	Mid-size facility capacity
Average IPD length of stay	4 days	→ ~24 new admissions/day
Registered staff accounts	180	Doctors, nurses, reception, pharmacy, lab, billing, store, admin
Daily active users	150 (≈83% of registered)	Shift-based attendance pattern
Peak concurrent users	45 (≈30% of DAU)	Morning OPD rush concentration
Operating window (primary load)	10 hours/day	8am–6pm, with lighter overnight IPD-only activity
Lab order rate	40% of OPD + IPD encounters	
Average lab parameters / test	5	
Read : write ratio (transactional)	4 : 1	Standard for clinical/administrative CRUD-and-lookup workloads
Real-time queue display refresh	5s (reception desk), 10s (OPD queue display)	Current front-end refresh configuration
Concurrent display/polling clients	4 reception desks + 4 OPD queue boards	Sized to a 120-bed / 600-OPD facility

4. User & Traffic Metrics

Metric	Value	Basis
Registered users	180	\$3
Daily active users	150	83% of registered
Peak concurrent users	~45	30% × DAU
Daily patient-facing volume	624 encounters (600 OPD + 24 IPD admissions)	\$3

5. Throughput — RPS / QPS

5.1 Write volume (transactional state changes)

Source	Calculation	Writes/day
OPD encounter lifecycle	600 visits × ~12 operations (registration, token issuance, diagnosis, prescription with items, dispensing, charge posting, receipt, closure)	7,200
IPD ongoing + admission workflow	96 occupied beds × 3/day + 24 admissions × 10 (allocation/assignment workflow)	528
Laboratory workflow	250 orders/day (40% of 624) × ~12 operations (order, items, sample, results, report)	3,000
Back-office (procurement/inventory)		100
Total		≈ 10,800/day

5.2 Read volume

Source	Calculation	Reads/day
Transactional reads	10,800 × 4 (read:write ratio)	43,200
Reception desk queue display	4 desks × (10hr × 3600s ÷ 5s refresh)	28,800
OPD queue board display	4 boards × (10hr × 3600s ÷ 10s refresh)	14,400
Total		≈ 86,400/day

5.3 Aggregate throughput

- **Total requests/day** ≈ 97,200
- **Average RPS** (across the 10-hour operating window) ≈ **2.7**
- **Peak RPS** (3–5× average during concentrated periods) ≈ **10–14**
- **Peak database QPS** (accounting for related-record query composition, ~3–4 queries per API request) ≈ **40–56**

At single-hospital scale this throughput profile is comfortably served by a single small application instance and a single database instance. Real-time display refresh accounts for a substantial share of total read volume; a push-based update mechanism (WebSocket/SSE) is an available optimization if read volume needs to be reduced at larger scale.

6. Storage Estimation

6.1 Per-encounter storage

Methodology: sum of the relevant Prisma model field counts × typical field byte size, with a 1.2–1.4× multiplier for index overhead, including the transaction audit trail associated with each encounter.

Component	OPD visit	IPD admission (4-day average)
Core clinical records (visit, diagnosis, prescription, orders)	≈2.6 KB	≈7.4 KB
Laboratory records	≈0.85 KB (probability-weighted)	≈4.3 KB
Billing records (charges, receipts)	≈1.6 KB	≈7.2 KB
Discharge documentation	—	≈1.5 KB
Clinical/billing subtotal (incl. index overhead)	≈6 KB	≈24 KB
Transaction audit trail	≈13 KB	≈78 KB
Total per encounter	≈19 KB	≈100 KB

6.2 Storage per registered patient (lifetime estimate)

Assuming an average of 3 clinical encounters/year (mixed OPD/IPD, weighted toward OPD) and a 5-year active relationship with the facility:

- Average annual storage per patient $\approx 3 \times 19 \text{ KB} \approx \mathbf{57 \text{ KB/year}}$
- 5-year cumulative storage per patient $\approx \mathbf{\approx 285 \text{ KB}}$

6.3 Annual database growth (single hospital)

Source	Volume/year	Storage/year
OPD	219,000 visits	≈4.0 GB
IPD	8,760 admissions	≈0.85 GB
Master/reference data (new registrations, catalog updates)		<10 MB
Total		≈4.8–5 GB/year

Projected 5-year database size (linear growth model): **≈25 GB per hospital.**

7. Bandwidth Estimation

Source	Calculation	MB/day
Real-time display refresh	43,200 requests × ~1 KB	~43
Standard API requests	~54,000 requests × ~4 KB	~220
Generated documents (receipts, reports; ~90 KB average)	~312/day × 90 KB	~27
Total		≈290 MB/day ≈ 8.7 GB/month per hospital

(Static frontend assets are served once per session via CDN/cache and are excluded from this per-request calculation.)

8. Memory Estimation

Component	Estimate
Application process baseline	150–250 MB
Document rendering engine (shared instance)	+300–500 MB
Per-concurrent-request overhead	+1–2 MB × concurrency
Recommended API instance memory	1.5–2 GB
Database <code>shared_buffers</code>	~25% of instance RAM
Recommended database instance memory (single hospital)	2–4 GB

9. Cache Sizing

Cacheable data	Estimated size
Role/permission matrix	<1 MB
Service, medicine, and lab catalogs (master data)	~1.5–6 MB
Facility branding configuration	<10 KB
Live OPD queue state	<100 KB
Total working set (single hospital)	well under 50 MB

A 256 MB cache instance provides substantial headroom for single-hospital operation; sizing should be revisited if catalog data becomes hospital-specific at multi-hospital scale (§11).

10. Latency Targets

Endpoint class	p50	p95	p99
Standard CRUD / lookup operations	<150 ms	<400 ms	<800 ms
Real-time queue display endpoints	<80 ms	<150 ms	<300 ms
Document generation (PDF receipts/reports)	~800 ms	<2.5 s	<4 s
Authentication	~120–200 ms	<350 ms	<500 ms

Document generation carries inherently higher latency due to full-page rendering; for high-concurrency scenarios this workload is a candidate for asynchronous processing (generate-and-notify rather than generate-and-block).

11. Replication, Reliability & Availability

11.1 Recommended production architecture

- **Database:** Primary instance with one streaming replica for failover and read-scaling; a dedicated read replica for analytics/reporting workloads once cross-hospital reporting is introduced (\$13), keeping heavy aggregation queries isolated from live operational traffic.
- **Backup:** Continuous write-ahead-log archiving to support point-in-time recovery, in addition to scheduled full backups.
- **Application tier:** Minimum two API instances behind a load balancer for redundancy once moving beyond initial pilot deployment.

11.2 Availability targets

Deployment phase	Target uptime	Recovery point objective	Recovery time objective
Initial single-hospital deployment	99.5% ($\approx$ 3.65 hrs/month)	$\leq$ 24 hours	$\leq$ 4 hours
Production with replica + automated failover	99.9% ($\approx$ 43 min/month)	Minutes (continuous WAL archiving)	$\leq$ 1 hour

12. Maintainability

- $\approx$ 35,200 LOC across 239 files and 21 backend domain modules — a scope consistent with single-team ownership and straightforward onboarding.
- Continuous integration validates lint, type-checking, and test suites on every change.
- Recommended additions ahead of scaled production operation: structured logging with centralized aggregation, application performance monitoring, and an automated deployment pipeline (build →

migrate → deploy) to standardize release operations as hospital count grows.

13. Multi-Hospital / Platform-Admin Expansion — Architecture Requirements

13.1 Objective

Introduce a platform-level administrative tier above the existing hospital `SuperAdmin` role, capable of managing multiple hospitals, each retaining its own `SuperAdmin` and subordinate role hierarchy.

13.2 Current data model

The schema is currently structured for single-hospital operation: all 59 models operate against one implicit facility context, and hospital branding is configured as a single global record. Extending to multiple hospitals introduces a tenancy dimension that the schema does not yet express.

13.3 Required extensions

1. **New `Hospital` entity** — id, name, code, address, branding configuration, active status.
2. **Hospital-scoping** on all hospital-specific records — employees, patients, visits and their downstream clinical/billing records, ward/room/bed inventory, and user accounts.
3. **Composite identifier scoping** for document numbering (receipt numbers, OPD tokens, lab order numbers, hospital UID codes) so identifiers remain unique per hospital rather than globally.
4. **Extended access-control model** — a platform-level scope (`PlatformAdmin` , cross-hospital) alongside the existing hospital-level scope (`HospitalSuperAdmin` and subordinate roles, single-hospital), with the authentication token carrying the active hospital context for every request.
5. **Centralized query scoping** — a consistent, systematic mechanism (e.g., a data-access layer or ORM middleware) ensuring every query is automatically scoped to the correct hospital context, rather than scoping being re-implemented per endpoint.
6. **Staged data migration** — backfilling hospital context onto existing records as the first hospital, applied in a sequenced migration plan.
7. **Cross-hospital reporting** — platform-level dashboards and aggregation queries routed to the dedicated analytics read replica (§11.1) to keep this workload isolated from per-hospital operational traffic.

13.4 Scale impact

Operational metrics (throughput, storage, bandwidth) scale approximately linearly with the number of participating hospitals, as shown in the Executive Summary scale table. Platform-level reporting represents a separate, lower-frequency query workload layered on top of the per-hospital operational load.

14. Appendix — Calculation Methodology

```
writes/day          =  $\Sigma(\text{encounter\_type\_count} \times \text{writes\_per\_encounter\_type})$ 
reads/day           =  $(\text{writes/day} \times \text{read:write\_ratio}) + \Sigma(\text{display\_clients} \times \text{operating\_seconds} \div$ 
refresh\_interval\_seconds)
avg_RPS              =  $\text{total\_requests/day} \div \text{operating\_window\_seconds}$ 
peak_RPS             =  $\text{avg\_RPS} \times \text{burst\_factor}$  (3-5x)
peak_QPS             =  $\text{peak\_RPS} \times \text{avg\_queries\_per\_request}$  (3-4)
storage/encounter    =  $\Sigma(\text{model\_field\_count} \times \text{avg\_field\_bytes}) \times \text{index\_overhead\_factor}$  (1.2-1.4x)
                    +  $(\text{writes\_per\_encounter} \times \text{audit\_row\_bytes} \times 1.2)$ 
storage/year         =  $\Sigma(\text{encounter\_type\_volume/year} \times \text{storage/encounter\_type})$ 
storage/patient      =  $\text{avg\_encounters\_per\_year} \times \text{storage/encounter} \times \text{relationship\_years}$ 
bandwidth/day        =  $\Sigma(\text{request\_class\_count/day} \times \text{avg\_payload\_bytes})$ 
platform\_scale(N)   =  $\text{per\_hospital\_metric} \times N$ , for hospital-scoped operational workload
```